



FLOWDIARY

AI-Powered Ethical Hacking

Lecture 7 — Web Technologies Overview

Front-end vs back-end · HTML / CSS / JavaScript · databases · APIs · common stacks

Tutor: Engr. Ibrahim Auwal

Why this lecture matters for ethical hackers

In Lecture 6 you traced how a request travels across the network. This lecture opens up what is actually running at each end — the technologies that build the web application itself.

Every layer here is an attack surface. Cross-site scripting lives in the front-end, SQL injection lives at the database, broken authentication lives in the back-end, and insecure APIs expose data directly. You cannot find a vulnerability in a system you do not understand. This lecture gives you that map.

Learning Objectives

By the end of this lecture you should be able to:

- Clearly distinguish front-end from back-end and explain what each is responsible for.
- Describe the roles of HTML, CSS, and JavaScript and how they work together.
- Explain what a database does and the difference between SQL and NoSQL.
- Explain what an API is, why it exists, and how REST APIs are structured.
- Recognize common technology stacks (LAMP, MERN, etc.) and what each part does.
- Identify where common web vulnerabilities live within these technologies.

1. Front-End vs Back-End

A web application has two halves that work together. The front-end is everything the user sees and interacts with in the browser. The back-end is everything that happens on the server, out of sight, to make the application actually work.

1.1 The front-end (client-side)

The front-end runs in the user's browser. It is responsible for presentation and interaction: the layout, the colors, the buttons, the forms, and the way things respond when you click or type. When developers say "client-side," they mean code that executes on the user's own device.

Because the front-end runs on the user's machine, anyone can view its source code. This is a crucial security fact: you should never trust the front-end to keep secrets or enforce rules, because the user controls it completely.

1.2 The back-end (server-side)

The back-end runs on the server. It handles the logic the user should not control: checking passwords, processing payments, deciding who is allowed to see what, and reading or writing to the database. When developers say "server-side," they mean code that executes on the server, where the user cannot see or change it.

Analogy: a restaurant, revisited

Front-end = the dining room. The menu, the table, the presentation of the plate — what the customer sees and touches.

Back-end = the kitchen and storeroom. The recipes, the ingredients, the rules about who can enter — hidden from the customer but where the real work happens.

Security lens: a clever customer can rewrite the menu (tamper with the front-end), but the kitchen must still verify every order independently. Trusting the dining room to enforce kitchen rules is how breaches happen.

1.3 How they communicate

The front-end and back-end talk over HTTP, usually by exchanging data through an API (covered in Section 4). The browser sends a request; the back-end does the work and sends data back; the front-end displays it. The two halves are developed differently but are constantly in conversation.

	Front-end	Back-end
Runs on	User's browser	Server
Visible to user?	Yes — fully inspectable	No — hidden
Responsible for	Presentation, interaction	Logic, data, security
Core technologies	HTML, CSS, JavaScript	Server language + database
Trust level	Never trust it	Enforce everything here

2. The Front-End Trio: HTML, CSS, JavaScript

Three technologies build every web page. They are often compared to the parts of a human body: HTML is the skeleton, CSS is the appearance, and JavaScript is the muscle that makes it move.

2.1 HTML — structure

HTML (HyperText Markup Language) defines the structure and content of a page: headings, paragraphs, links, images, forms, and buttons. It uses tags to label each piece of content so the browser knows what it is. HTML on its own is plain and unstyled — just organized content.

2.2 CSS — presentation

CSS (Cascading Style Sheets) controls how the HTML looks: colors, fonts, spacing, layout, and responsiveness across screen sizes. The same HTML can look completely different depending on its CSS. CSS separates appearance from content so each can change independently.

2.3 JavaScript — behavior

JavaScript makes pages interactive. It can respond to clicks, validate forms, update content without reloading, fetch data from the back-end, and run animations. It is the only one of the three that is a true programming language, and it runs inside the browser.

Security lens: the front-end trio

Cross-Site Scripting (XSS): if an app lets attacker-supplied text become live HTML or JavaScript on someone else's page, the attacker's code runs in the victim's browser — stealing sessions or keystrokes.

Client-side validation is not security: JavaScript form checks improve user experience, but an attacker can bypass them entirely. The back-end must re-validate everything.

Exposed logic: secrets hard-coded in front-end JavaScript (API keys, hidden endpoints) are visible to anyone who opens developer tools.

3. Databases — Where Data Lives

Most applications need to remember things: user accounts, posts, orders, settings. A database is the organized store where the back-end keeps this data so it survives between visits and can be searched quickly. The back-end talks to the database; the front-end never should directly.

3.1 SQL (relational) databases

SQL databases store data in tables — rows and columns, like a set of linked spreadsheets. They enforce a strict structure (a schema) and let you ask precise questions using a query language called SQL. They excel when data is well-structured and relationships matter, such as users linked to their orders. Examples include MySQL, PostgreSQL, and Microsoft SQL Server.

3.2 NoSQL (non-relational) databases

NoSQL databases store data more flexibly — as documents, key-value pairs, or graphs — without a rigid table structure. They are popular when data is unstructured or changes shape often, and when massive scale is needed. Examples include MongoDB (documents) and Redis (key-value).

	SQL (relational)	NoSQL (non-relational)
Structure	Tables, fixed schema	Flexible documents / pairs
Query language	SQL	Varies by database
Best for	Structured, related data	Unstructured, fast-changing data
Examples	MySQL, PostgreSQL	MongoDB, Redis

Security lens: SQL injection

SQL injection is one of the oldest and most damaging web vulnerabilities. It happens when user input is dropped directly into a database query instead of being safely separated from it. An attacker types crafted input that changes what the query does — letting them read, modify, or delete data they should never reach.

The fix: parameterized queries (prepared statements) keep user input as data, never as part of the command. This is a recurring theme — never mix untrusted input with trusted instructions.

4. APIs — How Systems Talk to Each Other

An API (Application Programming Interface) is a defined set of rules that lets two programs exchange data. When your front-end needs information from the back-end — or when an app needs data from another company's service — it goes through an API. The API is the agreed contract for what you can ask for and what you get back.

4.1 The waiter analogy

Think of an API as a waiter. You do not walk into the kitchen yourself; you give your order to the waiter, who carries it to the kitchen and brings back exactly what you asked for. You do not need to know how the kitchen works — only how to place an order the waiter understands. The API hides the complexity behind a simple, agreed interface.

4.2 REST APIs

The most common style of web API is REST. A REST API exposes resources (like users or products) at URLs called endpoints, and you act on them using HTTP methods. Data is usually exchanged in JSON, a lightweight, human-readable text format.

HTTP method	Action	Example
GET	Read data	Get a user's profile
POST	Create data	Create a new account
PUT / PATCH	Update data	Edit a profile
DELETE	Remove data	Delete a post

Security lens: APIs are a top target

Broken authentication / authorization: APIs often fail to check whether the caller is allowed to access a specific record — letting an attacker change an ID in the request and read someone else's data (IDOR).

Excessive data exposure: an API may return more fields than the front-end shows, leaking sensitive data to anyone inspecting the raw response.

Recon value: undocumented or hidden API endpoints are prime discovery targets — they often have weaker protection than the main site.

5. Putting It Together: Common Stacks

A "stack" is the combination of technologies used to build an application — from the database up to the front-end. Developers bundle proven combinations so the pieces are known to work well together. Knowing a target's stack tells an attacker which vulnerabilities and exploits are likely to apply.

5.1 What a stack contains

Most stacks have four kinds of component: an operating system or server environment, a back-end language/framework, a database, and a front-end. Naming a stack is just shorthand for listing these choices.

5.2 Well-known stacks

Stack	Components	Notes
LAMP	Linux, Apache, MySQL, PHP	Classic, very widely deployed.
MERN	MongoDB, Express, React, Node.js	All-JavaScript, popular modern stack.
MEAN	MongoDB, Express, Angular, Node.js	Like MERN but with Angular front-end.
Django	Python + Django + (often) PostgreSQL	Batteries-included Python framework.
.NET	C# + ASP.NET + SQL Server	Common in enterprise / Windows shops.

Security lens: fingerprinting the stack

Early in an assessment, an ethical hacker tries to identify the stack — from HTTP headers, error messages, cookie names, file extensions, or framework-specific behavior. Each technology has its own known weaknesses, default configurations, and version-specific exploits.

Lesson: revealing your exact stack and versions gives attackers a head start. Defenders reduce this signal; attackers work hard to uncover it.

6. The Big Picture

Here is how every concept in this lecture fits together in a single request, building directly on the journey from Lecture 6:

1. **The browser renders the front-end** using HTML (structure), CSS (style), and JavaScript (behavior).
2. **JavaScript calls an API** on the back-end to fetch or send data, usually as JSON over HTTP.
3. **The back-end runs server-side logic**, verifying permissions and validating input — the security checkpoint.
4. **The back-end queries the database** to read or store the data, using SQL or a NoSQL query.
5. **The response travels back** through the API to the front-end, which updates what the user sees.
6. **All of this runs on a stack** whose specific technologies determine the realistic threat picture.

Where the vulnerabilities live

Front-end → XSS, exposed secrets, bypassed client-side validation.

API → broken authorization, IDOR, excessive data exposure.

Back-end → broken authentication, insecure logic, misconfiguration.

Database → SQL/NoSQL injection, weak access control.

Knowing the layer tells you the class of bug to hunt for. This map is the foundation for the rest of the course.

7. Recap and Key Takeaways

- **Front-end is what users see** and can fully inspect; back-end is hidden and is where trust must be enforced.
- **HTML, CSS, JavaScript** are structure, style, and behavior — and never the place to enforce security.
- **Databases store the data**; SQL is structured and relational, NoSQL is flexible — and both can be injected.
- **APIs are the contract** between systems; REST over JSON is the common pattern and a major attack surface.
- **Stacks bundle these choices**; fingerprinting the stack guides which exploits apply.
- **Every layer maps to a vulnerability class** — knowing the architecture tells you where to look.

Discussion & Lab Prompts

1. Open developer tools on a website and inspect the HTML, CSS, and Network tab. Can you spot any API calls returning JSON?
2. Find a public REST API and make a GET request. Examine the JSON response — does it return more fields than a user would need?
3. Pick a website and try to fingerprint its stack from headers and behavior. What clues did you find?
4. Discuss: why is it a mistake to rely on JavaScript form validation for security? Give a concrete example of bypassing it.

Key Terms Glossary

Term	Quick definition
Front-end	Client-side code running in the browser.
Back-end	Server-side code the user cannot see.
HTML	Defines page structure and content.
CSS	Defines appearance and layout.
JavaScript	Adds interactivity and behavior.
Database	Organized store for persistent data.
SQL / NoSQL	Relational vs flexible data stores.
API	Rules letting two programs exchange data.
REST	Common API style using HTTP methods + JSON.
Stack	The full set of technologies in an app.